

UNIT -3(FUNCTIONS AND ARRAYS)

3.1.FUNCTIONS:

A function is a block of organized, reusable code that is used to perform a single, related action.

- Once a function is written, it can be reused as and when required. So, functions are also called reusable code.
- Functions provide modularity for programming.
- Code maintenance will become easy because of functions.
- When there is an error in the software, the corresponding function can be modified without disturbing the other functions in the software.
- The use of functions in a program will reduce the length of the program.

As you already know, Python gives you many built-in functions like `sqrt()`, etc. but you can also create your own functions. These functions are called *user-defined functions*.

Difference between a function and a method:

A function can be written individually in a python program. A function is called using its name. When a function is written inside a class, it becomes a „method“. A method is called using object name or class name. A method is called using one of the following ways:

Objectname.methodname()

Classname.methodname()

Defining a Function

You can define functions to provide the required functionality. Here are simple rules to define a function in Python.

- Function blocks begin with the keyword **def** followed by the function name and parentheses ().
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon (:) and is indented.
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return none`.

Syntax:

```
def functionname (parameters):  
    """function_docstring"""  
    function_suite  
    return [expression]
```

By default, parameters have a positional behavior and you need to inform them in the same order that they were defined.

Example:

```
def add(a,b):  
    """This function sum the numbers"""  
    c=a+b  
    print c  
    return
```

Calling Function:

A function cannot run by its own. It runs only when we call it. So, the next step is to call function using its name. While calling the function, we should pass the necessary values to the function in the parentheses as:

add(5,12)

Example:

```
def add(a,b):  
    """This function sum the numbers"""  
    c=a+b  
    print c  
add(5,12) # 17
```

3.2.Default Arguments:

We can mention some default value for the function parameters in the definition. Let's take the definition of grocery() function as:

def grocery(item, price=40.00)

Here, the first argument is „item“ whose default value is not mentioned. But the second argument is „price“ and its default value is mentioned to be 40.00. at the time of calling this function, if we do not pass „price“ value, then the default value of 40.00 is taken. If we mention the „price“ value, then that mentioned value is utilized. So, a default argument is an argument that assumes a default value if a value is not provided in the function call for that argument.

Example:

```
def grocery(item,price=40.00):  
    print "item=",item  
    print "price=",price  
grocery(item="sugar",price=50.75)  
grocery(item="oil")
```

Output:

```
item= sugar price= 50.75  
item= oil price= 40.0
```

Variable Length Arguments:

Sometimes, the programmer does not know how many values a function may receive. In that case, the programmer cannot decide how many arguments to be given in the function definition. For example, if the programmer is writing a function to add two numbers, he/she can write:

add(a,b)

But, the user who is using this function may want to use this function to find sum of three numbers. In that case, there is a chance that the user may provide 3 arguments to this function as:

add(10,15,20)

Then the add() function will fail and error will be displayed. If the programmer wants to develop a function that can accept „n“ arguments, that is also possible in python. For this purpose, a variable length argument is used in the function definition. A variable length argument is an argument that can accept any number of values. The variable length argument is written with a „*“ symbol before it in the function definition as:

def add(farg, *args):

here, „farg“ is the formal argument and „*args“ represents variable length argument. We can pass 1 or more values to this „*args“ and it will store them all in a tuple.

Example:

```
def add(farg,*args):
    sum=0
    for i in args:
        sum=sum+i
    print "sum is",sum+farg
add(5,10)
add(5,10,20)
add(5,10,20,30)
```

Output:

```
sum is 15
sum is 35
sum is 65
```

3.3. Anonymous Function or Lambdas:

These functions are called anonymous because they are not declared in the standard manner by using the *def* keyword. You can use the *lambda* keyword to create small anonymous functions.

- Lambda forms can take any number of arguments but return just one value in the form of an expression. They cannot contain commands or multiple expressions.
- An anonymous function cannot be a direct call to print because lambda requires an expression.
- Lambda functions have their own local namespace and cannot access variables other than

those in their parameter list and those in the global namespace.

- Although it appears that lambda's are a one-line version of a function, they are not equivalent to inline statements in C or C++, whose purpose is by passing function stack allocation during invocation for performance reasons.

Let's take a normal function that returns square of given value:

```
def square(x):  
    return x*x
```

the same function can be written as anonymous function as:

```
lambda x: x*x
```

The colon (:) represents the beginning of the function that contains an expression $x*x$. The syntax is:

```
lambda argument_list: expression
```

Example:

```
f=lambda x:x*x  
value = f(5) print  
value
```

The map() Function

The advantage of the lambda operator can be seen when it is used in combination with the map() function. map() is a function with two arguments:

```
r = map(func, seq)
```

Parameters :

fun : It is a function to which map passes each element of given iterable.

iter : It is a iterable which is to be mapped.

NOTE : You can pass one or more iterable to the map() function.

Returns :

Returns a list of the results after applying the given function to each item of a given iterable (list, tuple etc.)

EXAMPLE:

```
def addition(n):
```

```
    return n + n
```

```
numbers = (1, 2, 3, 4)
```

```
result = map(addition, numbers)
```

```
print(list(result))
```

Output :

[2, 4, 6, 8]

Filtering

The function `filter(function, list)` offers an elegant way to filter out all the elements of a list, for which the function *function* returns True. The function `filter(f,l)` needs a function *f* as its first argument. *f* returns a Boolean value, i.e. either True or False. This function will be applied to every element of the list *l*. Only if *f* returns True will the element of the list be included in the result list.

```
>>> fib = [0,1,1,2,3,5,8,13,21,34,55]
>>> result = filter(lambda x: x % 2, fib)
>>> print result
[1, 1, 3, 5, 13, 21, 55]
>>> result = filter(lambda x: x % 2 == 0, fib)
>>> print result
[0, 2, 8, 34]
```

Reducing a List

The function `reduce(func, seq)` continually applies the function `func()` to the sequence `seq`. It returns a single value.

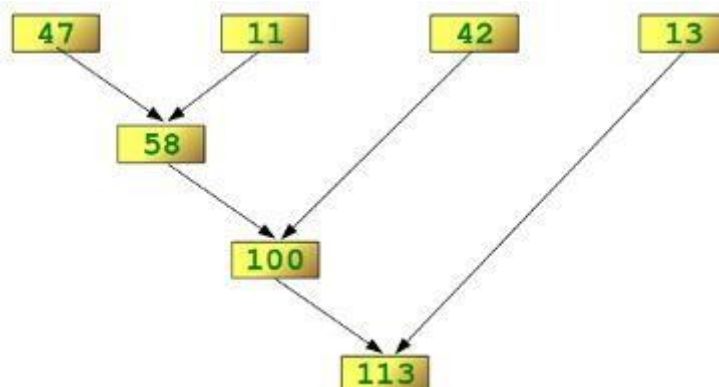
If `seq = [ s1, s2, s3, ... , sn ]`, calling `reduce(func, seq)` works like this:

- At first the first two elements of `seq` will be applied to `func`, i.e. `func(s1,s2)`. The list on which `reduce()` works looks now like this: `[ func(s1, s2), s3, ... , sn ]`
- In the next step `func` will be applied on the previous result and the third element of the list, i.e. `func(func(s1, s2),s3)`. The list looks like this now: `[ func(func(s1, s2),s3), ... , sn ]`
- Continue like this until just one element is left and return this element as the result of `reduce()`

We illustrate this process in the following example:

```
>>> reduce(lambda x,y: x+y, [47,11,42,13])
113
```

The following diagram shows the intermediate steps of the calculation:



Examples of reduce ()

Determining the maximum of a list of numerical values by using `reduce`:

```
>>> f = lambda a,b: a if (a > b) else b
>>> reduce(f, [47,11,42,102,13])
```

102

>>>

Calculating the sum of the numbers from 1 to 100:

```
>>> reduce(lambda x, y: x+y, range(1,101))
```

5050

3.4.Returning Results from a function:

We can return the result or output from the function using a „return“ statement in the function body. When a function does not return any result, we need not write the return statement in the body of the function.

Q) Write a program to find the sum of two numbers and return the result from the function.

```
def add(a,b):  
    """This function sum the numbers"""  
    c=a+b  
    return c  
print add(5,12) # 17  
print add(1.5,6) #6.5
```

Returning multiple values from a function:

A function can returns a single value in the programming languages like C, C++ and JAVA. But, in python, a function can return multiple values. When a function calculates multiple results and wants to return the results, we can use return statement as:

return a, b, c

Here, three values which are in „a“, „b“ and „c“ are returned. These values are returned by the function as a tuple. To grab these values, we can three variables at the time of calling the function as:

```
x, y, z = functionName( )
```

Here, „x“, „y“ and „z“ are receiving the three values returned by the function.

Example:

```
def calc(a,b):  
    c=a+b  
    d=a-b  
    e=a*b  
    return c,d,e  
x,y,z=calc(5,8)  
print "Addition=",x  
print "Subtraction=",y  
print "Multiplication=",z
```

\

3.5.Local and Global Variables:

When we declare a variable inside a function, it becomes a local variable. A local variable is a variable whose scope is limited only to that function where it is created. That means the local variable value is available only in that function and not outside of that function.

When the variable „a“ is declared inside myfunction() and hence it is available inside that function. Once we come out of the function, the variable „a“ is removed from memory and it is not available.

Example-1:

```
def myfunction():
    a=10
    print "Inside function",a #display 10
myfunction()
print "outside function",a # Error, not available
```

Output:

```
Inside function 10
outside function
```

NameError: name 'a' is not defined

When a variable is declared above a function, it becomes global variable. Such variables are available to all the functions which are written after it.

Example-2:

```
a=11
def myfunction():
    b=10
    print "Inside function",a #display global var
    print "Inside function",b #display local var
myfunction()
print "outside function",a # available
print "outside function",b # error
```

Output:

```
Inside function 11
Inside function 10
outside function 11
outside function
```

NameError: name 'b' is not defined

3.6.Modules:

A module is a file containing Python definitions and statements. The file name is the module name with the suffix.py appended. Within a module, the module's name (as a string) is available as the value of the global variable `__name__`. For instance, use your favourite text editor to create a file called fibo.py in the current directory with the following contents:

```
# Fibonacci numbers module
def fib(n): # write Fibonacci series up to na,
    b = 0, 1
```

```
while b < n:
    print b,
    a, b = b, a+b
def fib2(n): # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while b < n:
        result.append(b),
        b = b, a+b
    return result
```

3.7. importing modules:

Now enter the Python interpreter and import this module with the following command:

```
>>> import fibo
```

This does not enter the names of the functions defined in fibo directly in the current symbol table; it only enters the module name fibo there. Using the module name you can access the functions:

```
>>> fibo.fib(1000)
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.fib2(100)
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
>>> fibo.__name__
'fibo'
```

from statement:

- Modules can import other modules. It is customary but not required to place all import statements at the beginning of a module (or script, for that matter). The imported module names are placed in the importing module's global symbol table.
- There is a variant of the import statement that imports names from a module directly into the importing module's symbol table. For example:

```
>>> from fibo import fib, fib2
>>> fib(500)
1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

This does not introduce the module name from which the imports are taken in the local symbol table (so in the example, fibo is not defined).

There is even a variant to import all names that a module defines:

```
>>> from fibo import *
>>> fib(500)
1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

3.8.Namespaces and Scoping

- Variables are names (identifiers) that map to objects. A *namespace* is a dictionary of variable names (keys) and their corresponding objects (values).
- A Python statement can access variables in a *local namespace* and in the *global namespace*. If a local and a global variable have the same name, the local variable shadows the global variable.
- Each function has its own local namespace. Class methods follow the same scoping rule as ordinary functions.
- Python makes educated guesses on whether variables are local or global. It assumes

that any variable assigned a value in a function is local.

- Therefore, in order to assign a value to a global variable within a function, you must first use the global statement.
- The statement *global VarName* tells Python that *VarName* is a global variable. Python stops searching the local namespace for the variable.
- For example, we define a variable *Money* in the global namespace. Within the function *Money*, we assign *Money* a value, therefore Python assumes *Money* as a local variable. However, we accessed the value of the local variable *Money* before setting it, so an *UnboundLocalError* is the result. Uncommenting the global statement fixes the problem.

3.9.Packages in Python

We usually organize our files in different folders and subfolders based on some criteria, so that they can be managed easily and efficiently. For example, we keep all our games in a Games folder and we can even subcategorize according to the genre of the game or something like this. The same analogy is followed by the Python package.

A Python module may contain several classes, functions, variables, etc. whereas a Python package can contain several modules. In simpler terms a package is folder that contains various modules as files.

Creating Package

Let's create a package named *mypckg* that will contain two modules *mod1* and *mod2*. To create this module follow the below steps –

- Create a folder named *mypckg*.
- Inside this folder create an empty Python file i.e. *__init__.py*
- Then create two modules *mod1* and *mod2* in this folder.
- The hierarchy of the our package looks like this –

Mod1.py

```
def gfg():  
    print("Welcome to GFG")
```

Mod2.py

```
def sum(a, b):  
    return a+b
```

```
mypckg  
|  
|  
---__init__.py  
|  
|  
---mod1.py  
|  
|  
---mod2.py
```

Understanding *__init__.py*

__init__.py helps the Python interpreter to recognise the folder as package. It also specifies the resources to be imported from the modules. If the *__init__.py* is empty this means that all the functions of the modules will be imported. We can also specify the functions from each module to be made available.

Import Modules from a Package

We can import these modules using the from...import statement and the dot(.) operator.

Syntax:

```
import package_name.module_name
```

EXAMPLE:

```
from mypkg import mod1
from mypkg import mod2
mod1.gfg()
res = mod2.sum(1, 2)
print(res)
```

output:

Welcome to GFG
3

3.10.string slice and immutability.

How String slicing in Python works

For **understanding slicing** we will use different methods, here we will cover 2 methods of string slicing, the one is using the in-build slice() method and another using the **[:] array slice**. String slicing in Python is about obtaining a sub-string from the given string by slicing it respectively from start to end.

Python slicing can be done in two ways:

- Using a slice() method
- Using array slicing [: :] method

Index tracker for positive and negative index: String indexing and slicing in python. Here, the Negative comes into consideration when tracking the string in reverse.

0	1	2	3	4	5	6
A	S	T	R	I	N	G

-7	-6	-5	-4	-3	-2	-1
A	S	T	R	I	N	G

Method 1: Using slice() method

The slice() constructor creates a slice object representing the set of indices specified by range(start, stop, step).

Syntax:

- slice(stop)
- slice(start, stop, step)

Parameters: **start:** Starting index where the slicing of object starts. **stop:** Ending index where the slicing of object stops. **step:** It is an optional argument that determines the increment between each index for slicing. **Return Type:** Returns a sliced object containing elements in the given range only.

```
# String slicing

String ='ASTRING'

# Using slice constructor

s1 =slice(3)

s2 =slice(1, 5, 2)

s3 =slice(-1, -12, -2)

print(" String slicing ")

print(String[s1])

print(String[s2])

print(String[s3])
```

Output:

```
String slicing

AST

SR

GITA
```

Method 2: Using List/array slicing [: :] method

In Python, indexing syntax can be used as a substitute for the slice object. This is an easy and convenient way to slice a string using list slicing and Array slicing both syntax-wise and execution-wise. A start, end, and step have the same mechanism as the slice() constructor. Below we will see **string slicing in Python with example.**

Syntax

```
arr[start:stop]      # items start through stop-1
arr[start:]          # items start through the rest of the array
arr[:stop]           # items from the beginning through stop-1
arr[:]               # a copy of the whole array
arr[start:stop:step] # start through not past stop, by step
```

Example 1:

In this example, we will see **slicing in python list** the index start from 0 indexes and ending with a 2 index(stops at 3-1=2).s

```
String = "welcome"
```

```
print(String[:3])
```

Output:

wel

immutability

In python, the string data types are immutable. Which means a string value cannot be updated. We can verify this by trying to update a part of the string which will led us to an error.

```
# Can not reassign
t= "my python"
print type(t)
t[0] = "m"
```

When we run the above program, we get the following output –

```
t[0] = "M"
```

TypeError: 'str' object does not support item assignment

We can further verify this by checking the memory location address of the position of the letters of the string.

3.11.String functions and methods.

SNO	Method Name	Description
1	capitalize()	Capitalizes first letter of string.
2	center(width, fillchar)	Returns a space-padded string with the original string centered to a total of width columns.
3	count(str, beg= 0,end=len(string))	Counts how many times str occurs in string or in a substring of string if starting index beg and ending index end are given.
4	decode(encoding='UTF-8',errors='strict')	Decodes the string using the codec registered for encoding. Encoding defaults to the default string encoding.
5	encode(encoding='UTF-8',errors='strict')	Returns encoded string version of string; on error, default is to raise a Value Error unless errors is given with 'ignore' or 'replace'.
6	find(str, beg=0 end=len(string))	Determine if str occurs in string or in a substring of string if starting index beg and ending index end are given returns index if found and -1 otherwise.
7	index(str, beg=0, end=len(string))	Same as find(), but raises an exception if str not found.
8	isdigit()	Returns true if string contains only digits and false otherwise.

9	islower()	Returns true if string has at least 1 cased character and all cased characters are in lowercase and false otherwise.
10	isnumeric()	Returns true if a unicode string contains only numeric characters and false otherwise.
11	isspace()	Returns true if string contains only whitespace characters and false otherwise.
12	isupper()	Returns true if string has at least one cased character and all cased characters are in uppercase and false otherwise.
13	join(seq)	Merges (concatenates) the string representations of elements in sequence seq into a string, with separator string.
14	len(string)	Returns the length of the string.
15	ljust(width[, fillchar])	Returns a space-padded string with the original string left-justified to a total of width columns.
16	lower()	Converts all uppercase letters in string to lowercase.
17	max(str)	Returns the max alphabetical character from the string str.
18	min(str)	Returns min alphabetical character from the string str.
19	split(str="", num=string.count(str))	Splits string according to delimiter str (space if not provided) and returns list of substrings; split into at most num substrings if given.
20	splitlines (num=string.count('\n'))	Splits string at all (or num) NEWLINEs and returns a list of each line with NEWLINEs removed.
21	upper()	Converts lowercase letters in string to uppercase.
22	isdecimal()	Returns true if a unicode string contains only decimal characters and false otherwise.

:

EXAMPLE:

```
string = "Intellipaat"
print(len(string))
string = "Intellipaat Python program"
print(len(string))
a = "Intellipaat"
print(a[2:5])
a = "Intellipaat"
print(a[:5])
x = "intellipaat"[:-1]
print(x)
x="Intellipaat Python program"
a=x.split()
print(a)
a = "I like Programming"
b = a.replace("Programming", "Python")
print(b)
```

OUTPUT:

```
11
27
tel
Intel
taapilletni
['Intellipaat', 'Python', 'program']
I like Python
```

3.12.string module:

The string module provides additional tools to manipulate strings. Some methods available in the standard data structure are not available in the string module (e.g., isalpha).

```
>>> string.digits
'0123456789'
>>> string.punctuation
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
>>> string.lowercase
'abcdefghijklmnopqrstuvwxyz'
>>> string.letters
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>> string.uppercase
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

The following 3 function are equivalent to the previous 3 functions:

1. string.ascii_letters 2.string.ascii_lowercase3. string.ascii_uppercase

```
>>> string.expandtabs("a a\t",4)
'a a '
```

```
>>> string.whitespace
'\t\n\x0b\x0c\r '
```

```
>>> string.octdigits
'01234567'
```

Capitalise each word of the sentence:

```
>>> string.capwords("this is a test")
'This Is A Test'
```

string.splitfields equivalent to the split method of a string string.joinfields equivalent to the join method of a string

There are deprecated functions atof, atoi, atol

```
>>> string.atof("2")
2.0
```

equivalent to

```
>> float("2")
```

You also have the atoi (cast to integer) and atol (cast to long)

A function of interest is string.maketrans. Can be used together with the translate method of the string data structure.

Return a translation table suitable for passing to translate(), that will map each character in from into the character at the same position in to; from and to must have the same length.

```
>>> t = string.maketrans("acgt", "tgca")
>>> "acgttgca".translate(t)
'tgcaacgt'
```

The Formatter class in the string module allows you to create and customize your own string formatting behaviors using the same implementation as the built-in format() method.

3.13. Arrays:

An array is an object that stores a group of elements of same datatype.

- Arrays can store only one type of data. It means, we can store only integer type elements or only float type elements into an array. But we cannot store one integer, one float and

one character type element into the same array.

- Arrays can increase or decrease their size dynamically. It means, we need not declare the size of the array. When the elements are added, it will increase its size and when the elements are removed, it will automatically decrease its size in memory.

Creating an array:

Syntax:

```
arrayname = array(type code, [elements])
```

The type code „i“ represents integer type array where we can store integer numbers. If the type code is „f“ then it represents float type array where we can store numbers with decimal point.

Example:

The type code character should be written in single quotes. After that the elements should be written in inside the square braces [] as

```
a = array ( „i“, [4,8,-7,1,2,5,9] )
```

Importing the Array Module:

There are two ways to import the array module into our program.

The first way is to import the entire array module using import statement as,

```
import array
```

when we import the array module, we are able to get the „array“ class of that module that helps us to create an array.

```
a = array.array('i', [4,8,-7,1,2,5,9] )
```

Here the first „array“ represents the module name and the next „array“ represents the class name for which the object is created. We should understand that we are creating our array as an object of array class.

The next way of importing the array module is to write:

```
from array import *
```

Observe the „*” symbol that represents „all“. The meaning of this statement is this: import all (classes, objects, variables, etc) from the array module into our program. That means significantly importing the „array“ class of „array“ module. So, there is no need to mention the module name before our array name while creating it. We can create array as:

```
a = array('i', [4,8,-7,1,2,5,9] )
```

Example:

```
from array import *  
arr = array(„i“, [4,8,-7,1,2,5,9])  
for i in arr:  
    print i,
```

Output:

```
4 8 -7 1 2 5 9
```

3.14.accessing of an array:

Indexing and slicing of arrays:

An *index* represents the position number of an element in an array. For example, when we creating following integer type array:

Python programming –(302)

a = array('i', [10,20,30,40,50])

Python interpreter allocates 5 blocks of memory, each of 2 bytes size and stores the elements 10, 20, 30, 40 and 50 in these blocks.

10	20	30	40	50
a[0]	a[1]	a[2]	a[3]	a[4]

Example:

```
from array import *
a=array('i', [10,20,30,40,50,60,70])
print "length is",len(a)
print " 1st position character", a[1]
print "Characters from 2 to 4", a[2:5]
print "Characters from 2 to end", a[2:]
print "Characters from start to 4", a[:5]
print "Characters from start to end", a[:]

a[3]=45
a[4]=55
print "Characters from start to end after modifications ",a[:]
```

Output:

```
length is 7
1st position character 20
Characters from 2 to 4 array('i', [30, 40, 50])
Characters from 2 to end array('i', [30, 40, 50, 60, 70])
Characters from start to 4 array('i', [10, 20, 30, 40, 50])
Characters from start to end array('i', [10, 20, 30, 40, 50, 60, 70])
Characters from start to end after modifications array('i', [10, 20, 30, 45, 55, 60, 70])
```

3.15.Array Methods:

Method	Description
a.append(x)	Adds an element x at the end of the existing array a.
a.count(x)	Returns the number of occurrences of x in the array a.
a.extend(x)	Appends x at the end of the array a. „x“ can be another array or iterable object.
a.fromfile(f,n)	Reads n items from from the file object f and appends at the end of the array a.
a.fromlist(l)	Appends items from the l to the end of the array. l can be any list or iterable object.
a.fromstring(s)	Appends items from string s to end of the array a.
a.index(x)	Returns the position number of the first occurrence of x in the array. Raises „ValueError“ if not found.
a.pop(x)	Removes the item x from the array a and returns it.

a.pop()	Removes last item from the array a
a.remove(x)	Removes the first occurrence of x in the array. Raises „ValueError“ if not found.
a.reverse()	Reverses the order of elements in the array a.
a.tofile(f)	Writes all elements to the file f.
a.tolist()	Converts array „a“ into a list.
a.tostring()	Converts the array into a string.